

SOLID, DRY, KISS Principles

Personal Banking Management System
Software Design Principles Training

Training Objectives

- ✓ Master SOLID principles through practical examples
- ✓ Apply DRY (Don't Repeat Yourself) effectively
- ✓ Implement KISS (Keep It Simple, Stupid) methodology
- ✓ Build a maintainable Banking System

Why is learning design principles important?



Common Problems Without Good Design Principles

- Tightly coupled code that's hard to modify
- Duplicated logic across different modules
- Complex, hard-to-understand code
- Difficulty adding new account types or features
- Testing becomes nearly impossible

SOLID Principles Overview

S - Single Responsibility

A class should have only one reason to change

O - Open/Closed

Open for extension, closed for modification

L - Liskov Substitution

Subtypes must be substitutable for their base types

I - Interface Segregation

Many specific interfaces are better than one general purpose interface

D - Dependency Inversion

Depend on abstractions, not concretions

 **Remember:** SOLID principles help create maintainable, flexible, and testable code!

Single Responsibility Principle (SRP) - JAVA

✗ Violation Example

What is SRP?

"A class should have only one reason to change."

Each class should have a single responsibility and encapsulate only one part of the software's functionality.

Problems: This class has multiple responsibilities - account management, persistence, notifications, and reporting!



SRP_Violation.java



SRP_Complined.java

```
class BankAccount {
    private String accountNumber;
    private double balance;
    private String customerName;

    // --- Responsibility 1: Core Account Logic (Correct) ---
    public void deposit(double amount) {
        balance += amount;
    }

    public void withdraw(double amount) {
        balance -= amount;
    }

    // --- SRP VIOLATION 1: Persistence Logic ---
    public void saveToDatabase() { /* DB logic */ }
    public void loadFromDatabase() { /* DB logic */ }

    // --- SRP VIOLATION 2: Notification Logic ---
    public void sendEmailNotification() { /* Email logic */ }

    // --- SRP VIOLATION 3: Presentation/Reporting Logic ---
    public String generateStatement() {
        /* Report logic */
        return "";
    }
}
```

Single Responsibility Principle (SRP) - PYTHON

What is SRP?

"A class should have only one reason to change."

Each class should have a single responsibility and encapsulate only one part of the software's functionality.

Problems: This class has multiple responsibilities - account management, persistence, notifications, and reporting!

✗ Violation Example

```
class BankAccount:
    accountNumber: str
    customerName: str
    balance: float = 0.0
    _transactions: List[str] = field(default_factory=list)

    # --- Responsibility 1: Core Account Logic (Correct) ---
    def deposit(self, amount: float) -> None:
        self.balance += amount
        self._transactions.append(f"{datetime.now():%Y-%m-%d %H:%M} +{amount:.2f} -> {self.balance:.2f}")

    def withdraw(self, amount: float) -> None:
        self.balance -= amount
        self._transactions.append(f"{datetime.now():%Y-%m-%d %H:%M} -{amount:.2f} -> {self.balance:.2f}")

    # --- SRP VIOLATION 1: Persistence Logic ---
    def saveToDatabase(self) -> None: # stub for parity with Java
        pass

    def loadFromDatabase(self) -> None:
        pass

    # --- SRP VIOLATION 2: Notification Logic ---
    def sendEmailNotification(self) -> None:
        pass

    # --- SRP VIOLATION 3: Presentation/Reporting Logic ---
    def generateStatement(self) -> str:
        lines = [f"Statement for {self.customerName} ({self.accountNumber})",
                 "-" * 42]
        lines.extend(self._transactions)
        lines.append("-" * 42)
        lines.append(f"Current balance: {self.balance:.2f}")
        return "\n".join(lines)
```

SRP Violations Identified

● Violation 1: Persistence Logic

Methods: saveToDatabase(), loadFromDatabase()

Problem: What if the database changes from MySQL to MongoDB? The BankAccount class would need to change!

● Violation 2: Notification Logic

Methods: sendEmailNotification()

Problem: What if we want to send SMS instead of email? The BankAccount class would need to change!

● Violation 3: Presentation/Reporting Logic

Methods: generateStatement()

Problem: What if the statement format changes to PDF or we need different formats? The BankAccount class would need to change!

The Solution: Separate Responsibilities

✓ Refactored Design

- **BankAccount** - Only handles core account operations (deposit, withdraw, getBalance)
- **BankAccountRepository** - Handles database operations (save, load)
- **NotificationService** - Handles all notifications (email, SMS, push)
- **StatementGenerator** - Handles report generation (HTML, PDF, XML)

🎯 Benefits

- Each class has a single reason to change
- Easier to test individual components
- Better maintainability and extensibility
- Follows the Open-Closed Principle

Key Takeaways

✗ SRP Violations Lead To:

- Classes that are hard to maintain
- Changes ripple through unrelated functionality
- Difficult to test in isolation
- Tight coupling between different concerns

✓ Following SRP Results In:

- Highly cohesive classes
- Loosely coupled system
- Easy to extend and modify
- Better testability

"A class should have only one reason to change."



SRP_Violation.py



SRP_Complined.py

Open-Closed Principle

Strategy Pattern Solutions in Java & Python

! Core Problem: The OCP Violation

The fundamental issue: The code must be **modified** every time a new requirement is added. This often happens with long `if/else/else` or `switch` statements that check for a "type" and then execute logic.

 **Goal:** Add new functionality by writing **new code**, not by changing old, tested code.



Problematic Pattern

```
if (type.equals("SAVINGS")) {  
    // Savings logic  
    return balance * 0.02;  
} else if (type.equals("CHECKING")) {  
    // Checking logic  
    return balance * 0.001;  
} else if (type.equals("PREMIUM")) {  
    // Premium logic  
    return balance * 0.035;  
}  
// Every new type requires modifying this method!
```



The Solution: The Strategy Pattern

The most common solution is to use the **Strategy Pattern**. This involves three key steps:

1. **Define a Contract:** Create a common interface or protocol that all variations of the logic will follow.
2. **Implement Concrete Strategies:** Write separate classes for each variation, with each class implementing the contract.
3. **Use the Contract:** Make the main "calculator" or "service" class work with the contract, not the concrete classes. This decouples the main logic from the specific implementations.



Approach in Java

Java uses interfaces to enforce the contract and ensure type safety.

Key Concept: Java's static typing requires explicit interface implementation for compile-time safety.



OCP_Violation.java



OCP_Complined.java

Step 1: Define Interface Contract

This sets the rules. Every strategy must have a calculate() method.

```
public interface InterestStrategy {  
    double calculate(double balance);  
}
```

Step 2: Create Implementation Classes

Each class provides specific logic for one case.

```
public class SavingsStrategy implements InterestStrategy {  
    public double calculate(double balance) {  
        return balance * 0.02;  
    }  
}
```

Step 3: Use the Interface

The calculator works with any InterestStrategy object.

```
// The calculator works with any object that IS-A InterestStrategy  
InterestStrategy strategy = new SavingsStrategy();  
double interest = strategy.calculate(1000);
```

Approach in Python

Python is more flexible and often uses protocols and dictionaries for a dynamic approach.

Key Concept: Python uses "duck typing" and protocols for flexible, dynamic strategy implementation.



OCP_Violation.py



OCP_Complined.py

Step 1: Define Protocol Contract

This defines expected structure using "duck typing".

```
from typing import Protocol

class InterestStrategy(Protocol):
    def calculate(self, balance: float) -> float: ...
```

Step 2: Create Implementation Classes

No special 'implements' keyword needed with Protocol.

```
class SavingsStrategy:
    def calculate(self, balance: float) -> float:
        return balance * 0.02
```

Step 3: Use Dictionary Dispatch

A central service holds strategy mappings, avoiding if/elif chains.

```
# The service looks up the correct strategy and calls it.
rules = {"SAVINGS": SavingsStrategy()}
interest = rules["SAVINGS"].calculate(1000)
```



The Liskov Substitution Principle

Definition

The Liskov Substitution Principle is a core concept in object-oriented design that states if you have a class S that is a subtype of class T, you should be able to replace objects of type T with objects of type S without breaking the program.

In simple terms: A subclass should be a seamless replacement for its parent class.



Use Case: Personal Banking Management System

Let's model different types of bank accounts. We'll start with a base `BankAccount` class that has deposit and withdraw methods. We'll then create subtypes like `SavingsAccount` and a special `FixedTermDepositAccount`.



Key Concepts

What Causes Violations?

Violations often occur when a subclass overrides a parent method in a way that is unexpected, such as by throwing an exception it's not supposed to, doing nothing, or changing the fundamental behavior.

How to Spot Violations

Look for `UnsupportedOperationException`, empty implementations, or code that checks object types before calling methods.

The Core Problem

A `FixedTermDepositAccount` is a type of bank account, so inheriting from `BankAccount` seems logical. However, you cannot withdraw from a fixed-term account until it matures.

The Solution

Rethink the hierarchy! Create smaller, more specific interfaces. Use composition over inheritance when the "is-a" relationship doesn't truly fit.

LSP Violation Example - JAVA



LSP_Violation.java



LSP_Complied.java

⚠ The Problem

The FixedTermDepositAccount breaks the contract of its parent. Any function expecting a BankAccount would not anticipate a withdraw call to fail with an exception.

```
// Problem: LSP Violation in Java

// Base class with a clear contract: can deposit and withdraw.
class BankAccount {
    protected double balance;

    public void deposit(double amount) {
        balance += amount;
    }

    public void withdraw(double amount) {
        if (balance >= amount) {
            balance -= amount;
        } else {
            throw new IllegalArgumentException("Insufficient funds");
        }
    }
}

// This subtype follows the contract perfectly.
class SavingsAccount extends BankAccount {
    // Inherits methods as-is.
}

// This subtype BREAKS the contract.
class FixedTermDepositAccount extends BankAccount {
    @Override
    public void withdraw(double amount) {
        // Violates LSP by throwing an unexpected exception
        throw new UnsupportedOperationException(
            "Cannot withdraw from a fixed-term deposit account.");
    }
}
```

✓ The Solution

Remodel the inheritance hierarchy. Don't force a withdraw method on accounts that cannot support it. Create a more specific abstraction for withdrawable accounts.

```
// Solution: Refactoring to comply with LSP

// A base class for all accounts, only with common operations.
abstract class Account {
    protected double balance;

    public void deposit(double amount) {
        balance += amount;
    }
}

// An interface for accounts that specifically support withdrawals.
interface Withdrawable {
    void withdraw(double amount);
}

// SavingsAccount supports both depositing and withdrawing.
class SavingsAccount extends Account implements Withdrawable {
    @Override
    public void withdraw(double amount) {
        if (balance >= amount) {
            balance -= amount;
        } else {
            throw new IllegalArgumentException("Insufficient funds");
        }
    }
}

// FixedTermDepositAccount only supports depositing, which is fine.
class FixedTermDepositAccount extends Account {
    // No withdraw method, so no LSP violation.
}
```



LSP_Violation.py



LSP_Complied.py

LSP Violation Example - PYTHON



The Problem

```
class BankAccount:
    def __init__(self, balance: float = 0.0) -> None:
        self._balance = balance

    def deposit(self, amount: float) -> None:
        self._balance += amount

    def withdraw(self, amount: float) -> None:
        if self._balance < amount:
            raise ValueError("Insufficient funds")
        self._balance -= amount

    @property
    def balance(self) -> float:
        return self._balance

class SavingsAccount(BankAccount):
    # Inherits behavior as-is (OK)
    pass

class FixedTermDepositAccount(BankAccount):
    def withdraw(self, amount: float) -> None:
        # ✗ LSP violation: unexpected exception for a core behavior
        raise NotImplementedError("Cannot withdraw from a fixed-term deposit account.")

def process_withdrawal(acct: BankAccount, amount: float) -> None:
    # Expects any BankAccount to support withdraw()
    acct.withdraw(amount) # Will explode for FixedTermDepositAccount

if __name__ == "__main__":
    savings = SavingsAccount()
    savings.deposit(200)
    process_withdrawal(savings, 50) # OK

    fixed = FixedTermDepositAccount()
    fixed.deposit(500)
    try:
        process_withdrawal(fixed, 100) # ✗ NotImplementedError
    except Exception as e:
        print("Caught:", e)
    print("Balances -> savings:", savings.balance, "| fixed:", fixed.balance)
```



The Solution

```
class Account(ABC):
    def __init__(self, balance: float = 0.0) -> None:
        self._balance = balance
    def deposit(self, amount: float) -> None:
        self._balance += amount
    @property
    def balance(self) -> float:
        return self._balance

class Withdrawable(ABC):
    @abstractmethod
    def withdraw(self, amount: float) -> None:
        ...

class SavingsAccount(Account, Withdrawable):
    def withdraw(self, amount: float) -> None:
        if self._balance < amount:
            raise ValueError("Insufficient funds")
        self._balance -= amount

class FixedTermDepositAccount(Account):
    # No withdraw() - not withdrawable until maturity
    pass

def withdraw_if_possible(obj, amount: float) -> None:
    if isinstance(obj, Withdrawable):
        obj.withdraw(amount)
    else:
        print(f"Not withdrawable type: {obj.__class__.__name__}")

if __name__ == "__main__":
    savings = SavingsAccount()
    savings.deposit(300)
    withdraw_if_possible(savings, 120) # OK

    fixed = FixedTermDepositAccount()
    fixed.deposit(1000)
    withdraw_if_possible(fixed, 100) # Gracefully skipped

    print("Balances -> savings:", savings.balance, "| fixed:", fixed.balance)
```



Key Takeaways

🔥 How to Spot LSP Violations

- **UnsupportedOperationException:** A subclass method throws an exception the parent doesn't
- **Empty implementations:** A subclass method does nothing
- **Type checking:** Code checks object type before calling methods

✅ How to Fix Violations

- **"Tell, Don't Ask":** Don't ask for object type, tell it what to do
- **Rethink Hierarchy:** The "is-a" relationship might be wrong
- **Specific Interfaces:** Create role-based interfaces (Withdrawable, Depositable)

🔧 Design Principles

- **Interface Segregation:** Break down large classes into smaller, focused interfaces
- **Composition over Inheritance:** Sometimes objects should contain behavior, not inherit it
- **Contract Compliance:** Subclasses must honor parent contracts

💎 Best Practices

- **Behavioral Compatibility:** Subclasses should strengthen postconditions, not weaken them
- **Precondition Rules:** Don't strengthen preconditions in subclasses
- **Semantic Consistency:** Maintain the meaning of operations across the hierarchy



Remember

"If it looks like a duck, quacks like a duck, but needs batteries — you probably have the wrong abstraction!"

Interface Segregation Principle

Building Clean, Focused Interfaces

"Clients should not be forced to depend on interfaces they do not use. It's better to have many small, specific interfaces than one large, general-purpose one."

 *It's like ordering a combo meal but being forced to take items you're allergic to. ISP lets you order just what you want.*



Core ISP Concepts



Core Idea

No client should be forced to implement interface methods it doesn't use

Avoid "fat interfaces" that bundle unrelated functionality

Keep interfaces focused and cohesive



Goal

Avoid forcing classes to implement irrelevant methods

Keep the system decoupled and clean

Improve maintainability and readability



Use Case: Banking System

Imagine a single, large `BankingOperations` interface with methods for every possible action: `deposit()`, `withdraw()`, `openFixedDeposit()`, `applyForLoan()`, `requestCreditCardStatement()`

✗ The Problem: ISP Violation - JAVA

🚨 Fat Interface Violation

```
// ISP_Violation.java
interface BankingOperations {
    void checkBalance(String accountId);
    void makeDeposit(String accountId, double amount);
    void makeWithdrawal(String accountId, double amount);
    void openNewAccount(String customerName);
    void closeAccount(String accountId);
}

class AccountHolder implements BankingOperations {
    @Override public void checkBalance(String accountId) { System.out.println("Checking balance..."); }
    @Override public void makeDeposit(String accountId, double amount) { System.out.println("Depositing..."); }
    @Override public void makeWithdrawal(String accountId, double amount) { System.out.println("Withdrawing..."); }
    @Override public void openNewAccount(String customerName) { throw new UnsupportedOperationException("Account holders cannot open new ac
    @Override public void closeAccount(String accountId) { throw new UnsupportedOperationException("Account holders cannot close accounts."
}

public class ISP_Violation {
    public static void main(String[] args) {
        BankingOperations customer = new AccountHolder();
        customer.checkBalance("ACC123");
        try { customer.openNewAccount("John"); } catch (UnsupportedOperationException ex) {
            System.out.println("Violation demonstrated: " + ex.getMessage());
        }
    }
}
```

✓ In summary:

The violation here is that `AccountHolder` is forced to implement operations (`openNewAccount` , `closeAccount`) that do not apply to its role, breaking the Interface Segregation Principle.

❌ The Problem: ISP Violation - PYTHON

🚨 Fat Interface Violation

```
from typing import Protocol

class BankingOperations(Protocol):
    def check_balance(self, account_id: str) -> None: ...
    def make_deposit(self, account_id: str, amount: float) -> None: ...
    def make_withdrawal(self, account_id: str, amount: float) -> None: ...
    def open_new_account(self, customer_name: str) -> None: ...
    def close_account(self, account_id: str) -> None: ...

class AccountHolder(BankingOperations):
    def check_balance(self, account_id: str) -> None:
        print("Checking balance...")
    def make_deposit(self, account_id: str, amount: float) -> None:
        print("Depositing...")
    def make_withdrawal(self, account_id: str, amount: float) -> None:
        print("Withdrawing...")
    def open_new_account(self, customer_name: str) -> None:
        raise NotImplementedError("Account holders cannot open new accounts.")
    def close_account(self, account_id: str) -> None:
        raise NotImplementedError("Account holders cannot close accounts.")
```

✅ In summary:

The violation here is that `AccountHolder` is forced to implement operations (`openNewAccount`, `closeAccount`) that do not apply to its role, breaking the Interface Segregation Principle.

✖ Problem & Impact



Problems Created

Empty Methods: Forced to implement irrelevant functionality

Tight Coupling: Classes depend on methods they don't use

Poor Maintainability: Changes affect unrelated classes



Impact

Confusing Code: UnsupportedOperationException everywhere

Brittle Design: Adding features breaks existing code

Testing Complexity: Must test irrelevant methods



The Solution: Segregated Interfaces - JAVA

Segregated Interfaces

```
// Small, focused interfaces
interface Transactional {
    void deposit(double amount);
    void withdraw(double amount);
}

interface Loanable {
    void applyForLoan(double amount);
}

interface Statementable {
    void requestCreditCardStatement();
}

interface Investable {
    void openFixedDeposit(double amount);
}
```

Clean Implementation

```
// SavingsAccount only implements what it needs
class SavingsAccount implements Transactional {
    private double balance;

    public void deposit(double amount) {
        balance += amount;
    }

    public void withdraw(double amount) {
        balance -= amount;
    }
    // No irrelevant methods!
}

// CreditAccount implements multiple relevant interfaces
class CreditAccount implements
    Transactional, Loanable, Statementable {
    // Implements only what it actually needs
}
```



ISP_Violation.java



ISP_Complied.java



Extension Example

```
// Adding new functionality is easy and doesn't break existing code
class BusinessAccount implements Transactional, Loanable, Investable {
    // Implements only the interfaces it needs
    // No forced implementation of irrelevant methods
}
```



The Solution: Segregated Interfaces - PYTHON

Segregated Interfaces

CodeMate | Qodo Gen: Options | Test this class

```
class AccountHolderActions(Protocol):  
    def check_balance(self, account_id: str) -> None: ...  
    def make_deposit(self, account_id: str, amount: float) -> None: ...  
    def make_withdrawal(self, account_id: str, amount: float) -> None: ...
```

CodeMate | Qodo Gen: Options | Test this class

```
class BankTellerActions(Protocol):  
    def open_new_account(self, customer_name: str) -> None: ...  
    def close_account(self, account_id: str) -> None: ...  
    def make_deposit(self, account_id: str, amount: float) -> None: ...
```

Clean Implementation

CodeMate | Qodo Gen: Options | Test this class

```
class AccountHolderActions(Protocol):  
    def check_balance(self, account_id: str) -> None: ...  
    def make_deposit(self, account_id: str, amount: float) -> None: ...  
    def make_withdrawal(self, account_id: str, amount: float) -> None: ...
```

CodeMate | Qodo Gen: Options | Test this class

```
class AccountHolder:  
    Qodo Gen: Options | Test this method  
    def check_balance(self, account_id: str) -> None:  
        print("Checking balance...")  
    Qodo Gen: Options | Test this method  
    def make_deposit(self, account_id: str, amount: float) -> None:  
        print("Depositing...")  
    Qodo Gen: Options | Test this method  
    def make_withdrawal(self, account_id: str, amount: float) -> None:  
        print("Withdrawing...")
```

CodeMate | Qodo Gen: Options | Test this function

```
def perform_customer_actions(client: AccountHolderActions, account_id: str):  
    client.check_balance(account_id)
```



ISP_Violation.py



ISP_Complied.py

🌟 Benefits of ISP Compliance

🌟 High Cohesion

Classes only implement methods that are relevant to their purpose, making them more focused and easier to understand.

🌟 Low Coupling

Changes in one interface don't affect classes that don't use it. Loan changes won't require recompiling SavingsAccount.

🌟 Improved Readability

Clear separation of concerns makes the codebase easier to navigate and understand for new developers.

🌟 Enhanced Maintainability

The system becomes safer to change with reduced risk of breaking unrelated functionality.

🌟 Flexible Design

Classes can implement multiple small interfaces as needed, creating flexible and composable designs.

🌟 Better Testing

Focused interfaces lead to more targeted unit tests and easier mocking for test scenarios.



Dependency Inversion Principle

Creating Loosely Coupled & Resilient Software

Part 1

High-level modules should not depend on low-level modules. Both should depend on abstractions (e.g., interfaces).

Part 2

Abstractions should not depend on details. Details (concrete implementations) should depend on abstractions.



Simple Terms

Don't let important business logic (high-level) depend directly on specific, technical details (low-level). Instead, make both depend on a common contract or interface.

✗ The Problem: DIP Violation

✗ Java Implementation

```
// Low-level module: A specific implementation
public class EmailNotifier {
    public void sendEmail(String message) {
        System.out.println("Sending email: " + message);
        // Logic to send an email
    }
}

// High-level module: Contains business logic
public class TransactionService {
    // Direct dependency on concrete low-level class
    private EmailNotifier emailNotifier = new EmailNotifier();

    public void completeTransaction(double amount) {
        // ... transaction logic ...
        System.out.println("Transaction completed.");
        emailNotifier.sendEmail("Transaction successful.");
    }
}
```

✗ Why This Is Problematic

Tight Coupling

High-level policy (notifying about transactions) becomes tightly coupled to the low-level mechanism (sending an email).

Rigidity

What if the bank decides to switch to SMS notifications or add push notifications? You would have to modify TransactionService every time.

Hard to Test

Cannot easily mock or substitute the notification mechanism for testing.

Java Solution: Following DIP



DIP_Violation.java



DIP_Complined.java

✓ Step 1: Create Abstraction

```
// Step 1: Create an abstraction (the contract)
public interface Notifier {
    void send(String message);
}
```

✓ Step 2: Implement Concrete Classes

```
// Step 2: Make low-level modules implement the abstraction
public class EmailNotifier implements Notifier {
    @Override
    public void send(String message) {
        System.out.println("Sending email: " + message);
    }
}

public class SmsNotifier implements Notifier {
    @Override
    public void send(String message) {
        System.out.println("Sending SMS: " + message);
    }
}
```

✓ Step 3: Depend on Abstraction

```
// Step 3: Make high-level module depend only on abstraction
public class TransactionService {
    private final Notifier notifier; // Depends on interface!

    // Dependency injection via constructor
    public TransactionService(Notifier notifier) {
        this.notifier = notifier;
    }

    public void completeTransaction(double amount) {
        // ... transaction logic ...
        System.out.println("Transaction completed.");
        notifier.send("Transaction successful."); // Uses abstraction!
    }
}
```



Python Solution: Using Protocols



DIP_Violation.pyDIP_Complined.py



✗ Why This Is Problematic

🔗 Tight Coupling

High-level policy (notifying about transactions) becomes tightly coupled to the low-level mechanism (sending an email).

💔 Rigidity

What if the bank decides to switch to SMS notifications or add push notifications? You would have to modify TransactionService every time.

🔧 Hard to Test

Cannot easily mock or substitute the notification mechanism for testing.

✗ Violating DIP

```
# Low-level module
class EmailNotifier:
    def send_email(self, message: str):
        print(f"Email: {message}")

# High-level module
class TransactionService:
    def __init__(self):
        # Direct dependency!
        self.notifier = EmailNotifier()

    def complete_transaction(self, amount: float):
        print(f"Transaction completed")
        self.notifier.send_email(f"Success!")
```

✓ Following DIP

```
from typing import Protocol

# Step 1: Create abstraction
class Notifier(Protocol):
    def send(self, message: str) -> None: ...

# Step 2: Concrete implementations
class EmailNotifier:
    def send(self, message: str) -> None:
        print(f"Email: {message}")

class SmsNotifier:
    def send(self, message: str) -> None:
        print(f"SMS: {message}")

# Step 3: Depend on abstraction
class TransactionService:
    def __init__(self, notifier: Notifier):
        self.notifier = notifier

    def complete_transaction(self, amount: float):
        print(f"Transaction completed")
        self.notifier.send(f"Success!")
```



Benefits of Following DIP



Why DIP Makes Your Code Better



Flexibility

Easily swap implementations (EmailNotifier ↔ SmsNotifier) without changing business logic.



Maintainability

Changes are isolated and don't cascade through the system. Modify notification logic without touching transaction logic.



Testability

Easily "mock" dependencies (e.g., TestNotifier) to test TransactionService in isolation.



Loose Coupling

Modules are independent and can evolve separately. High-level policy is protected from low-level changes.



Extensibility

Add new notification types (PushNotifier, SlackNotifier) without modifying existing code.



Single Responsibility

Each class has a clear, focused purpose. Transaction logic and notification logic are separated.



Key Takeaway

"Depend upon abstractions, not concretions" - This principle inverts the typical dependency structure, making your architecture more flexible and maintainable.

DRY Principle

Don't Repeat Yourself

"Every piece of knowledge must have a single, unambiguous, authoritative representation within a system."

— The Pragmatic Programmer



Use Case: Personal Banking System

Customer Information Validation

Building a banking system where we need to validate customer information. A common requirement is ensuring email and phone number formats are correct before creating or updating customer profiles.



Single Source of Truth

One place to define validation logic



Easy Maintenance

Change once, apply everywhere



Fewer Bugs

No inconsistent implementations

❌ The Violation: Repeated Logic - JAVA

🔥 Duplicated Validation Logic

```
public class CustomerService {  
    public void createCustomer(String name, String email, String phone) {  
        // 🔴 Repeated validation block  
        if (email == null || !email.contains("@")) {  
            throw new IllegalArgumentException("Invalid email format.");  
        }  
        if (phone == null || phone.length() != 10) {  
            throw new IllegalArgumentException("Invalid phone number format.");  
        }  
        System.out.println("Creating customer: " + name);  
    }  
  
    public void updateCustomer(int customerId, String email, String phone) {  
        // 🔴 Same validation block repeated!  
        if (email == null || !email.contains("@")) {  
            throw new IllegalArgumentException("Invalid email format.");  
        }  
        if (phone == null || phone.length() != 10) {  
            throw new IllegalArgumentException("Invalid phone number format.");  
        }  
        System.out.println("Updating customer: " + customerId);  
    }  
}
```

Java



DRY_Violation.java



The Solution: Abstract the Logic - JAVA

✦ Private Helper Method Approach

```
public class CustomerService {  
    // ✅ Single validation method  
    private void validateContactInfo(String email, String phone) {  
        if (email == null || !email.contains("@")) {  
            throw new IllegalArgumentException("Invalid email format.");  
        }  
        if (phone == null || phone.length() != 10) {  
            throw new IllegalArgumentException("Invalid phone number format.");  
        }  
    }  
  
    public void createCustomer(String name, String email, String phone) {  
        validateContactInfo(email, phone); // ✅ Reuse  
        System.out.println("Creating customer: " + name);  
    }  
  
    public void updateCustomer(int customerId, String email, String phone) {  
        validateContactInfo(email, phone); // ✅ Reuse  
        System.out.println("Updating customer: " + customerId);  
    }  
}
```

Java



DRY_Complied.java

❌ The Violation: Repeated Logic - PYTHON

🔔 Duplicated Validation Logic

```
class CustomerService:
    def createCustomer(self, name: str, email: str, phone: str) -> None:
        # Repeated validation block (❌)
        if not email or "@" not in email:
            raise ValueError("Invalid email format.")
        if not phone or len(phone) != 10:
            raise ValueError("Invalid phone number format.")
        print(f"Creating customer: {name}")
        # ... database logic to save new customer

    def updateCustomer(self, customerId: int, email: str, phone: str) -> None:
        # Repeated validation block (❌)
        if not email or "@" not in email:
            raise ValueError("Invalid email format.")
        if not phone or len(phone) != 10:
            raise ValueError("Invalid phone number format.")
        print(f"Updating customer: {customerId}")
        # ... database logic to update customer

if __name__ == "__main__":
    svc = CustomerService()
    svc.createCustomer("Alice", "alice@example.com", "1234567890")
    svc.updateCustomer(42, "bob@example.com", "0987654321")
```

Python



DRY_Violation.py



The Solution: Abstract the Logic - PYTHON

✦ Private Helper Method Approach

```
def validateContactInfo(email: str, phone: str) -> None:
    if not email or "@" not in email:
        raise ValueError("Invalid email format.")
    if not phone or len(phone) != 10:
        raise ValueError("Invalid phone number format.")

class CustomerService:
    def createCustomer(self, name: str, email: str, phone: str) -> None:
        validateContactInfo(email, phone) # ☑ single source of truth
        print(f"Creating customer: {name}")
        # ... database logic

    def updateCustomer(self, customerId: int, email: str, phone: str) -> None:
        validateContactInfo(email, phone) # ☑ single source of truth
        print(f"Updating customer: {customerId}")
        # ... database logic

# Optional: shared util module could hold validateContactInfo for use across services.

if __name__ == "__main__":
    svc = CustomerService()
    svc.createCustomer("Charlie", "charlie@example.com", "1112223333")
    svc.updateCustomer(7, "dana@example.com", "4445556666")
```

Python



DRY_Complied.py



Benefits of Following DRY



Maintainability

Change validation rules in one place, not scattered across multiple methods



Consistency

Identical logic everywhere prevents subtle bugs from variations



Testability

Test validation logic once, not in every method that uses it



Efficiency

Faster development and fewer lines of code to maintain

Key Takeaway

Identify → Abstract → Reuse

KISS Principle

Keep It Simple, Stupid

The KISS principle states that most systems work best if they are kept simple rather than made **complicated**. Simplicity should be a key goal in design, and unnecessary complexity should be avoided. In software, this means writing clear, straightforward, and easily understandable code. 💡

🎯 Use Case: Personal Banking Management System

Requirement: Calculate a withdrawal fee. The rule is simple: a \$5 fee is applied if the account balance is less than \$500. Otherwise, the withdrawal is free.

Java Implementation Java

The Violation (Over-engineered Code)

This code is unnecessarily complex. It uses nested conditions, a temporary variable for the fee, and a boolean flag, making it hard to read at a glance.

VIOLATION: Complicated logic for a simple rule



```
// VIOLATION: Complicated logic for a simple rule
public double calculateWithdrawalFee(double balance, double withdrawalAmount) {
    double fee = 0.0;
    boolean isFeeApplicable = false;

    if (withdrawalAmount > 0) {
        if (balance < 500) {
            isFeeApplicable = true;
        } else {
            isFeeApplicable = false;
        }
    }

    if (isFeeApplicable) {
        fee = 5.0;
    }

    // More logic could be here...
    return fee;
}
```

Java



KISS_Violation.java

Java Solution Java

✓ The Solution (Simple & Clear)

This version is direct and immediately understandable. It clearly states the single condition for applying the fee.

✓ COMPLIANT: Simple, direct, and easy to read

```
// COMPLIANT: Simple, direct, and easy to read
public double calculateWithdrawalFee(double balance) {
    if (balance < 500) {
        return 5.0; // Apply the fee
    }
    return 0.0; // No fee
}

// An even more concise version using a ternary operator
public double calculateWithdrawalFeeTernary(double balance) {
    return (balance < 500) ? 5.0 : 0.0;
}
```

Java



KISS_Complined.java

Java Key Takeaways ☕ Java



✓ Key Improvements Made:

- **Avoid Unnecessary Variables:** Removed `isFeeApplicable` and the temporary fee variable.
- **Reduce Nesting:** Flattened the nested if statements.
- **Return Early:** Returned the value as soon as the decision was made, which simplifies the code flow.
- **Focus on the Core Condition:** The logic now directly reflects the business rule: "a fee is applied if the balance is less than \$500."

Python Implementation Python

The Violation (Over-engineered Code)

This code uses a dictionary to mimic a switch statement and includes redundant checks, which is not idiomatic or simple for this problem.

 VIOLATION: Overly complex for a simple binary choice 

```
# VIOLATION: Overly complex for a simple binary choice
def calculate_withdrawal_fee(balance: float, withdrawal_amount: float) -> float:
    fee_status = "NO_FEE"

    if withdrawal_amount > 0:
        if balance < 500:
            fee_status = "APPLY_FEE"

# Using a dictionary as a switch is overkill here
    fee_map = {
        "APPLY_FEE": 5.0,
        "NO_FEE": 0.0
    }

    return fee_map.get(fee_status, 0.0)
```



KISS_Violation.py

Python Solution Python

✓ The Solution (Simple & Pythonic)

This version is clean, readable, and uses a simple conditional expression, which is a common and clear pattern in Python.

✓ COMPLIANT: Simple, direct, and Pythonic

```
# COMPLIANT: Simple, direct, and Pythonic
def calculate_withdrawal_fee(balance: float) -> float:
    """Calculates a $5 withdrawal fee if the balance is below $500."""
    if balance < 500:
        return 5.0
    return 0.0

# An even more concise "one-liner" version
def calculate_withdrawal_fee_online(balance: float) -> float:
    """Calculates a $5 withdrawal fee if the balance is below $500."""
    return 5.0 if balance < 500 else 0.0
```



KISS_Complined.py

Python Key Takeaways Python



Key Improvements Made:

- **Use Direct Conditionals:** Replaced the complex dictionary lookup with a simple if/else.
- **Embrace Readability:** The simplified code reads almost like plain English.
- **Leverage Conditional Expressions:** Python's one-line if/else is perfect for simple binary choices.
- **Avoid Premature Generalization:** Don't build a complex structure (like the `fee_map`) for a simple problem. Solve the immediate problem in the simplest way possible.

Remember KISS

Simplicity is the Ultimate Sophistication

Simple code is:

- Easier to read and understand
- Faster to debug and maintain
- Less prone to bugs
- More testable
- Better for team collaboration



*"Perfection is achieved not when there is nothing more to add,
but when there is nothing left to take away."*
— Antoine de Saint-Exupéry

DTOs vs. Entities

Decoupling Your API from Your Database

Java

Use Case: Personal Banking Management System

Core Principle: An Entity represents your internal system of record (the database), while a DTO represents the data contract for your external clients (the API). 💡



Entity



Internal database representation with business logic and persistence annotations



DTO



External API contract with clean, tailored data for specific use cases

What is an Entity?



The Internal Truth

Java

An Entity is a rich domain object that directly maps to a database table. It represents the core business data and contains business logic. 

 **Purpose:** To be the single source of truth for data within your application and database.

 Contains:

- Primary keys and foreign keys
- Sensitive data that should never be exposed (e.g., hashed passwords, SSN)
- Complex object graphs and relationships to other entities
- JPA/Persistence annotations (@Entity, @Table, @Id, @Column, @OneToMany)

Banking Example: Customer Entity

Java

This class mirrors the customers' table in your database.

Customer Entity - Database Representation

```
@Entity
@Table(name = "customers")
public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String fullName;

    @Column(unique = true, nullable = false)
    private String socialSecurityNumber; // Sensitive Data!

    @OneToMany(mappedBy = "customer")
    private List<Account> accounts; // Complex Relationship

    // Business Logic, getters, setters...
}
```



Entities_Example.java

What is a DTO?



The Public Contract

Java

A **Data Transfer Object (DTO)** is a **simple, plain object** used to transfer data between different layers of your application, especially between the server and the client (e.g., via a REST API). 

 **Purpose:** To act as a clean, tailored "data contract" for a specific use case, decoupling the client from your internal database structure.

 Contains:

- Only the fields required for a specific API endpoint or view
- No business logic, no database annotations
- Flattened data structures, easy for clients to consume
- Validation annotations (@NotNull, @Size) and JSON formatting (@JsonProperty)

Banking Example: CustomerDTO

Java

This DTO is designed to show basic, non-sensitive customer info for a public profile view.

CustomerDTO - API Contract

```
public class CustomerDTO {  
    @JsonProperty("customerName")  
    private String fullName;  
  
    @JsonProperty("memberSince")  
    private LocalDate registrationDate;  
  
    private int activeAccounts;  
  
    // Only getters and setters... No business logic!  
}
```



DTO_Example.java

The Core Strategy: Why Separate Them?

Building Robust, Secure, and Maintainable Systems

Java



Security

You prevent accidentally exposing sensitive data. The Customer entity has an SSN, but the CustomerDTO does not. You control exactly what goes out over the wire.



API Stability (Decoupling)

You can change your internal database schema (the Entity) without breaking your public API contract (the DTO). Database changes don't affect client applications.



Optimized Payloads

Clients often need a subset or aggregation of data. A DTO can be shaped perfectly for a specific UI component, preventing over-fetching and under-fetching.



Clear Boundaries

It enforces a clean separation of concerns. The Service Layer maps between Entities and DTOs, creating clear boundaries between domain logic and presentation.

Practical Flow in the Banking System

Scenario: A mobile app fetches a customer's profile

Java

1 **Request:** The client sends a GET request to `/api/customers/{id}`

2 **Controller:** The `CustomerController` receives the request

3 **Service Layer (The Magic Happens Here ✨):**

- The `CustomerService` calls the `CustomerRepository` to fetch the `Customer` entity from the database:

```
Customer customerEntity = customerRepository.findById(id);
```

- The service then maps the `Customer` entity to a `CustomerDTO`. This is a critical step:

```
CustomerDTO dto = mapEntityToDto(customerEntity);
```

4 **Controller:** The controller receives the `CustomerDTO` from the service

Practical Flow in the Banking System

Java

Scenario: A mobile app fetches a customer's profile

4 **Controller:** The controller receives the CustomerDTO from the service

5 **Response:** The controller returns the CustomerDTO, which gets serialized into a clean JSON object for the client

Resulting JSON sent to the client:

```
{ "customerName": "Jane Doe", "memberSince": "2023-05-10", "activeAccounts": 2 }
```

Notice the `socialSecurityNumber` and the complex `accounts` list are nowhere to be found! 🧐



Pydantic vs. ORM Models

A Decoupled Approach for Robust APIs

Use Case: Personal Banking Management System

Key Takeaway: Separate your concerns for robust and maintainable APIs

Core Concepts - What Are They?

Python



Pydantic Models

The "API Layer"

Purpose: Define the shape of external data. They act as the public-facing contract for your API.

Key Function: Data validation, serialization (Python to JSON), and deserialization (JSON to Python).

Analogy: Think of them as a customs declaration form ensuring everything entering or leaving your application is well-formed, typed, and valid.



ORM Models

The "Database Layer"

Purpose: Define the structure of your database tables. They map Python objects directly to database rows.

Key Function: Database interaction (Create, Read, Update, Delete - CRUD operations).

Analogy: Think of them as the blueprint for a bank vault defining exactly how data is structured and stored securely.

Why Use Both? The Power of Decoupling

Python

Keeping Pydantic and ORM models separate is a best practice for several key reasons:

Security

You never expose your internal database structure directly to the outside world. Your API can have a different shape than your database tables, preventing accidental data leaks.

Flexibility

Your API can evolve independently of your database schema. You can add a field to your API response without needing to add a new column to your database, and vice-versa.

Clarity

Each model has a single, clear responsibility. Pydantic models handle API validation. ORM models handle database state.

Validation Control

You can have different validation rules for incoming data (e.g., password must be 8 characters) versus what's stored in the database (e.g., a hashed password).

The Pattern - How They Interact

Python

The typical workflow follows a clear, one-way data flow, managed by a "Service" or "Business Logic" layer.

1 API Endpoint (/create_customer)

Receives raw JSON data from an HTTP request. Uses a Pydantic model (CustomerCreate) to parse, validate, and type-cast this incoming data.

2 Service Layer (customer_service.py)

Receives the validated Pydantic model object. Contains the business logic (e.g., check if email exists, hash the password). Maps the data from the Pydantic model to an ORM model (CustomerDB).

3 Database Layer (crud.py)

Receives the populated ORM model object. Commits the ORM model to the database, creating a new row.

4 Response

The newly created data from the database (in the ORM model) is converted back into a Pydantic model (CustomerPublic) to be sent as a JSON response, ensuring sensitive data (like password hashes) is excluded.

Use Case - Creating a New Bank Customer

Python

Let's apply this pattern to adding a new customer to our Personal Banking Management System.

Requirement:

An API endpoint to create a new customer with a name, email, and password. The password must be stored securely, and the API response should not include the password.

Pydantic Models

The API Contract

CustomerCreate: For incoming data. Requires name, email, and a password.

CustomerPublic: For outgoing data. Includes id, name, and email, but omits the password.

ORM Model

The Database Blueprint

CustomerDB: Represents the customers table. Has columns for id, name, email, and hashed_password.

Code Example - Putting It All Together

Python

1. Pydantic Models (schemas.py)

```
from pydantic import BaseModel, EmailStr

# For incoming data -> The "Create" schema
class CustomerCreate(BaseModel):
    name: str
    email: EmailStr
    password: str

# For outgoing data -> The "Public" schema
class CustomerPublic(BaseModel):
    id: int
    name: str
    email: EmailStr

class Config:
    from_attributes = True # Allows creating from ORM model
```



schemas.py

2. ORM Model (models.py)

```
from sqlalchemy import Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()

# Represents the database table
class CustomerDB(Base):
    __tablename__ = "customers"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, index=True)
    email = Column(String, unique=True, index=True)
    hashed_password = Column(String)
```



models.py

Service Layer Implementation

Python

3. Service Layer Logic (service.py)

```
# Fictional hashing function
def get_password_hash(password: str) -> str:
    return f"hashed_{password}"

# Business logic that connects the two models
def create_customer(customer_data: CustomerCreate) -> CustomerDB:
    # 1. Hash the password from the Pydantic model
    hashed_password = get_password_hash(customer_data.password)

    # 2. Create an ORM model instance
    db_customer = CustomerDB(
        name=customer_data.name,
        email=customer_data.email,
        hashed_password=hashed_password
    )

    # 3. Save db_customer to the database (not shown)
    # db.add(db_customer)
    # db.commit()
    # db.refresh(db_customer)

    return db_customer
```



service.py

Key Point: The service layer acts as the bridge between your API and database, handling business logic and data transformation.



Dependency Injection (DI)

Building Flexible & Testable Code

The "Hollywood Principle":
"Don't call us, we'll call you" - Inversion of Control (IoC)

Core Idea: A design pattern where a class receives its dependencies from an external source rather than creating them itself.

Analogy:

Think of a car. A car doesn't build its own engine. The engine (a dependency) is built separately and is injected into the car during assembly.

Main Goal:

To achieve **Loose Coupling** between classes, making your code more modular, flexible, and easier to test.

It's a form of a broader principle called Inversion of Control (IoC).

Understanding DI Through Analogy

Think of a Chef in a Kitchen

✗ Without DI

The chef has to:

- 🌱 Grow their own vegetables
- 🐄 Raise their own cattle
- ⌚ Do everything from scratch every time
- 🛠️ Manage the entire supply chain

Result: Chef is overwhelmed and can't focus on cooking!

✓ With DI

A supplier delivers:

- 🌱 Fresh ingredients (dependencies)
- 🍖 Quality meat products
- 📦 Everything ready to use
- 🔄 Easy to swap suppliers if needed

Result: Chef focuses solely on cooking excellence!

Why Use Dependency Injection?



Decoupling

Classes are not tightly bound to their dependencies. A service that sends notifications doesn't need to know *how* the notification is sent (Email, SMS, etc.), only that it has a "notifier" that can send a message.



Enhanced Testability

You can easily "mock" or fake dependencies during testing. To test a transaction service, you can inject a `MockNotificationService` that prints to console instead of sending real emails or SMS.



Flexibility & Maintainability

Easy to swap out components. If your bank decides to switch from emails to SMS messages, you just inject a different notification service without changing core transaction logic.

The Problem: Tight Coupling

Without DI, classes create their own dependencies, leading to a rigid and fragile system.

Use Case:

An AccountService needs to save data and send notifications.

✗ Tightly Coupled Code - AVOID THIS!

```
public class AccountService {  
  
    // The service creates its own dependencies directly.  
    private final AccountRepository repository = new AccountRepository();  
    private final NotificationService notificationService = new EmailNotificationService();  
  
    public void createAccount(String customerName) {  
        System.out.println("Creating account for " + customerName);  
        repository.save(customerName);  
        notificationService.sendNotification("Welcome, " + customerName);  
    }  
}
```

Problem 1 (Testing):

You can't test AccountService without also testing the real AccountRepository and EmailNotificationService. You can't use mock or fake objects.

Problem 2 (Flexibility):

What if you want to switch from EmailNotificationService to SmsNotificationService? You must change the code inside AccountService.

The Problem: Banking System Without DI 👎

Python

✗ Tightly Coupled Dependencies

The TransactionService creates its own dependencies directly, leading to rigid and fragile code.

✗ Bad Practice - Tightly Coupled Code

```
# Tightly coupled dependencies
class EmailNotifier:
    def send(self, message: str):
        print(f"EMAIL Sent: {message}")

class TransactionService:
    def __init__(self):
        # The service CREATES its own dependency. This is bad!
        self._notifier = EmailNotifier()

    def process_transaction(self, amount: float):
        # ... logic to process the transaction ...
        self._notifier.send(f"Processed transaction of {amount}")

# Usage
service = TransactionService()
service.process_transaction(100.0)
```

Problems with This Approach:

- 🔒 **Testing Issues:** You can't test TransactionService without also testing the real EmailNotifier. No mock objects possible!
- 🚫 **Inflexibility:** Want to switch to SMS? You must change the TransactionService code itself.

The Solution: Constructor Injection (Preferred Method)

Dependencies are provided through the class's constructor. This is the most common and recommended approach.

✓ Loosely Coupled with Constructor Injection - DO THIS!

```
public class AccountService {  
  
    // Dependencies are declared but not instantiated.  
    private final AccountRepository repository;  
    private final NotificationService notificationService;  
  
    // The dependencies are "injected" via the constructor.  
    public AccountService(AccountRepository repo, NotificationService notifier) {  
        this.repository = repo;  
        this.notificationService = notifier;  
    }  
  
    public void createAccount(String customerName) {  
        System.out.println("Creating account for " + customerName);  
        repository.save(customerName);  
        notificationService.sendNotification("Welcome, " + customerName);  
    }  
}
```

JAVA

Benefit:

The AccountService is no longer responsible for creating its dependencies. It just states what it needs to function.

Guarantees:

The object is always created in a valid, usable state with all its required dependencies present.

The Solution: Banking System With DI 👍

Python

✅ Loosely Coupled with Constructor Injection

Dependencies are injected through the constructor, making the code flexible and testable.

✅ Good Practice - Dependency Injection

```
# Define the dependencies
class EmailNotifier:
    def send(self, message: str):
        print(f"EMAIL Sent: {message}")

class SmsNotifier:
    def send(self, message: str):
        print(f"SMS Sent: {message}")

# --- The Refactored Service ---
class TransactionService:
    def __init__(self, notifier):
        # The dependency is INJECTED, not created.
        self._notifier = notifier

    def process_transaction(self, amount: float):
        # ... logic to process the transaction ...
        self._notifier.send(f"Processed transaction of {amount}")
```

Another Method: Setter Injection

Dependencies are provided through public setter methods after the object has been created.

Loosely Coupled with Setter Injection

```
public class AccountService {  
  
    private AccountRepository repository;  
    private NotificationService notificationService;  
  
    // Default constructor is empty.  
    public AccountService() {}  
  
    // Dependencies are "injected" via setter methods.  
    public void setRepository(AccountRepository repository) {  
        this.repository = repository;  
    }  
  
    public void setNotificationService(NotificationService notificationService) {  
        this.notificationService = notificationService;  
    }  
  
    // ... createAccount method ...  
}
```

✓ Use Case:

Best for optional dependencies that are not critical for the object's initial state.

⚠ Drawback:

The object can exist in an incomplete state before its dependencies are set.

Methods of Dependency Injection

Python

Constructor Injection (Preferred)

When: Dependencies are passed through the class constructor

Best for: Required dependencies

Benefit: Ensures object is created in valid state

```
class AccountService:
    def __init__(self, repo, notifier):
        self.repo = repo
        self.notifier = notifier
```

Setter Injection

When: Dependencies are set via setter methods after creation

Best for: Optional dependencies

Drawback: Object can exist in incomplete state

```
class AccountService:
    def set_repository(self, repo):
        self.repo = repo

    def set_notifier(self, notifier):
        self.notifier = notifier
```

Putting It All Together (The "Injector")

An external part of your application is responsible for creating dependencies and injecting them.

Putting it all together — The "Injector"

```
public class DI_PersonalBankingDemo {
    public static void main(String[] args) {
        System.out.println("=== Tight Coupling (avoid) ===");
        TightlyCoupledAccountService bad = new TightlyCoupledAccountService();
        bad.createAccount("Ria Sharma");

        System.out.println("\n=== Constructor Injection (preferred) ===");
        // 1) Create dependencies (can be switched without changing AccountService)
        AccountRepository repo = new DatabaseAccountRepository();
        NotificationService email = new EmailNotificationService();
        // NotificationService sms = new SmsNotificationService(); // easy swap
        // 2) Inject into the service
        AccountService good = new AccountService(repo, email);
        // 3) Use the service
        good.createAccount("Ria Sharma");

        System.out.println("\n=== Setter Injection ===");
        AccountServiceWithSetters withSetters = new AccountServiceWithSetters();
        withSetters.setRepository(new DatabaseAccountRepository());
        withSetters.setNotificationService(new SmsNotificationService()); // show easy swap
        withSetters.createAccount("Ria Sharma");

        System.out.println("\n=== Testing with a Mock (benefit of DI) ===");
        MockNotificationService mock = new MockNotificationService();
        AccountService testable = new AccountService(new DatabaseAccountRepository(), mock);
        testable.createAccount("Test User");
        System.out.println("Mock captured messages: " + mock.getSentMessages());
    }
}
```



DI_PersonalBankingDemo.java

- 1 Create all required dependencies first
- 2 Inject dependencies into the main service
- 3 Use the fully configured service

Python

Flexible Usage Examples

Flexible Usage - Same Service, Different Dependencies

```
# --- Flexible Usage ---
# Injecting an EmailNotifier
email_notifier = EmailNotifier()
email_service = TransactionService(notifier=email_notifier)
email_service.process_transaction(250.0)
# Output: EMAIL Sent: Processed transaction of 250.0

# Injecting an SmsNotifier without changing the service!
sms_notifier = SmsNotifier()
sms_service = TransactionService(notifier=sms_notifier)
sms_service.process_transaction(500.0)
# Output: SMS Sent: Processed transaction of 500.0

# For testing - Mock notifier
class MockNotifier:
    def send(self, message: str):
        print(f"MOCK: {message}")

mock_notifier = MockNotifier()
test_service = TransactionService(notifier=mock_notifier)
test_service.process_transaction(1000.0)
# Output: MOCK: Processed transaction of 1000.0
```



dependency_injection_banking.py

 **Key Insight:** The same TransactionService works with different notifiers without any code changes to the service itself!

Key Takeaways

Decoupling

Classes are independent. AccountService doesn't know or care which NotificationService it's using, as long as it fulfills the contract.

Easier Testing

You can easily "inject" mock or fake objects during tests. For example, provide a MockNotificationService to verify method calls without actually sending emails.

Improved Reusability

Components can be easily swapped and reused in different contexts. Same service, different implementations.

Better Maintainability

Code is cleaner, more organized, and easier to understand and modify. You change configurations, not code.

Remember: Dependency Injection promotes the principle of "Don't call us, we'll call you" - let the framework or container manage your dependencies!



Building Your First Web API with Spring Boot

Understanding POJOs, DTOs, and Dependency Injection

JAVA



Classroom Assignment



Today's Mission - The Project Task

Goal: Build a "Fund Transfer" API

What it does: Allows a user to move money between their own accounts.

Key Steps:

- 📄 Model the data (POJOs & DTOs)
- ⚙️ Write the business logic (The Service)
- 🌐 Expose it to the world (The REST Controller)
- 🔗 Connect them cleanly (Dependency Injection)

🕒 Time Limit: 30 Minutes - Let's make it count!



What is a POJO? (Plain Old Java Object)

A simple, ordinary Java object. No special powers!

- It's not tied to any framework (like Spring)
- **Purpose:** To represent your core business data
- Think of it as the "noun" of your application
- **In our case:** A BankAccount is a perfect POJO

```
// This is a POJO. Simple fields and methods.

public class BankAccount {
    private String accountNumber;
    private String accountHolderName;
    private double balance;

    // Constructors, Getters, Setters...
    public BankAccount(String accountNumber, String accountHolderName, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
    }
}
```



What is a DTO? (Data Transfer Object)

An object designed specifically to carry data between processes

- **Purpose:** To shape the data for a specific need, like an API request or response
- It acts as a contract for your API
- It decouples your internal domain model (POJO) from your external-facing API
- **In our case:** We need an object to represent the fund transfer request

```
// This is a DTO. Only the data needed for this specific action.
```

```
public class FundTransferRequest {  
    private String fromAccountNumber;  
    private String toAccountNumber;  
    private double amount;  
  
    // Getters, Setters...  
    public String getFromAccountNumber() { return fromAccountNumber; }  
    public void setFromAccountNumber(String fromAccountNumber) {  
        this.fromAccountNumber = fromAccountNumber;  
    }  
    // ... other getters and setters  
}
```

POJO vs. DTO - Why Both?

POJO (BankAccount)

Represents: The complete, internal state of our data
It might have fields or methods that the outside world should never see

Scope: Internal domain model

DTO (FundTransferRequest)

Represents: Only the data needed for one specific action
It's a clean, simple contract for our API endpoint

Scope: External API communication

Key Benefit: Separation of Concerns

Your internal data model can evolve without breaking your public API contract!

✖ The Problem: Tight Coupling

Without DI, classes create their own dependencies, leading to a rigid and fragile system.

// Tightly Coupled Code - AVOID THIS!

```
public class AccountService {  
    // The service creates its own dependencies directly.  
    private final AccountRepository repository = new AccountRepository();  
    private final NotificationService notificationService = new EmailNotificationService();  
  
    public void createAccount(String customerName) {  
        System.out.println("Creating account for " + customerName);  
        repository.save(customerName);  
        notificationService.sendNotification("Welcome, " + customerName);  
    }  
}
```

🚫 Problem 1 - Testing

You can't test AccountService without also testing the real AccountRepository and EmailNotificationService. No mocking possible!

🚫 Problem 2 - Flexibility

Want to switch from EmailNotificationService to SmsNotificationService? You must change the code inside AccountService.



The Solution: Constructor Injection

Dependencies are provided through the class's constructor. This is the most common and recommended approach.

```
// Loosely Coupled with Constructor Injection - DO THIS!
```

```
public class AccountService {  
    // Dependencies are declared but not instantiated.  
    private final AccountRepository repository;  
    private final NotificationService notificationService;  
  
    // The dependencies are "injected" via the constructor.  
    public AccountService(AccountRepository repo, NotificationService notifier) {  
        this.repository = repo;  
        this.notificationService = notifier;  
    }  
  
    public void createAccount(String customerName) {  
        System.out.println("Creating account for " + customerName);  
        repository.save(customerName);  
        notificationService.sendNotification("Welcome, " + customerName);  
    }  
}
```

✦ **Benefit:** The AccountService is no longer responsible for creating its dependencies

🔵 **Guarantees:** The object is always created in a valid, usable state



What is Dependency Injection in Spring?

Analogy:

A chef doesn't build their own oven. They just say, "I need an oven in my kitchen," and the restaurant owner (the framework) provides one!

Bad Way

`MyController controller = new MyController(new MyService());`
The controller creates its own service. They are tightly coupled!

The Solution (DI)

A class declares the dependencies it needs, and Spring provides them automatically!

In Spring, it's as simple as using the `@Autowired` annotation!



Connecting the Pieces with DI

Our AccountController needs the AccountService to do the actual work. Instead of creating it, we just ask for it!

```
@RestController
@RestController
public class AccountController {

    // Spring, please give me an instance of AccountService!
    @Autowired
    private AccountService accountService;

    @PostMapping("/transfer")
    public void transferFunds(@RequestBody FundTransferRequest request) {
        // The controller uses the service it was given.
        accountService.transferFunds(request);
    }
}
```

Controller



Service



Repository



Complete Solution Structure

AccountService.java (The Business Logic)

```
@Service
public class AccountService {

    @Autowired
    private AccountRepository accountRepository;

    public void transferFunds(FundTransferRequest request) {
        BankAccount fromAccount = accountRepository
            .findByAccountNumber(request.getFromAccountNumber())
            .orElseThrow(() -> new RuntimeException("Source account not found"));

        BankAccount toAccount = accountRepository
            .findByAccountNumber(request.getToAccountNumber())
            .orElseThrow(() -> new RuntimeException("Destination account not found"));

        if (fromAccount.getBalance() < request.getAmount()) {
            throw new RuntimeException("Insufficient funds");
        }

        // Perform the transfer
        fromAccount.setBalance(fromAccount.getBalance() - request.getAmount());
        toAccount.setBalance(toAccount.getBalance() + request.getAmount());

        accountRepository.save(fromAccount);
        accountRepository.save(toAccount);
    }
}
```



Let's Get Building!



Recap of the Task:



BankingApplication_Java.zip

Create **BankAccount** (POJO) and **FundTransferRequest** (DTO)

Create **AccountService** with a transferFunds method

Create **AccountController** with a POST endpoint

Use **@Autowired** to connect the Controller and Service

Add **@Service**, **@Repository**, **@RestController** annotations

I AM A WORK IN PROGRESS



You have 30 minutes. Let's build something amazing!

Remember: Focus on structure and relationships, not database complexity!



Key Benefits of Using DI

 **Decoupling:** Classes are independent. AccountService doesn't know or care which NotificationService it's using.

 **Easier Testing:** You can easily "inject" mock or fake objects during tests.

 **Improved Reusability:** Components can be easily swapped and reused in different contexts.

 **Better Maintainability:** Code is cleaner, more organized, and easier to understand.



The Bottom Line

You change configurations, not code. Your application becomes flexible and maintainable!

🏁 Conclusion & Key Takeaways

POJOs

Are for your internal domain model. They represent your core business entities like BankAccount.

DTOs

Are the contract for your API. They shape data for external communication, providing flexibility and security.

Dependency Injection

Is about Inversion of Control. It lets Spring manage the lifecycle of your objects (@Service, @Repository). It makes your code decoupled, modular, and much easier to test.

Congratulations!

You've just built a robust, scalable service structure that is fundamental to modern application development. Well done! 🎉



Building APIs in Python

The Python Developer's Guide

Python



light_banking_api.zip

Implementing Spring Patterns with FastAPI, Pydantic & SQLAlchemy

Goal: Show that while syntax and libraries change, the principles of building clean, layered APIs are universal

FastAPI

Spring Boot equivalent

Pydantic

Bean Validation + DTOs

SQLAlchemy

JPA/Hibernate equivalent



Today's Challenge (in 30 Minutes!)

Your Mission: Implement the "Create Bank Account" feature

Build a complete feature from API endpoint down to the database, exactly as you would in a layered Java application

Controller/API Layer

Receives HTTP requests
Handles validation & routing

Service Layer

Executes business logic
Orchestrates operations

Persistence Layer

Interacts with database
Manages data operations

Feature Specification:

- **API Endpoint:** POST /accounts/
- **Request Data:** account_type, initial balance, customer_id
- **Business Logic:** Generate unique account number
- **Database:** Save to bank_accounts table
- **Response:** Return complete account details with new ID



The Core Pattern: Python vs. Java

The pattern is identical. Only the names of the tools change.

Concept	Java / Spring Boot	Python / FastAPI
API Data Carrier	DTO (Plain Old Java Object)	Schema (Pydantic BaseModel)
Database Model	@Entity (JPA)	ORM Model (SQLAlchemy)
Business Logic	@Service Class	Plain Python Class
API Endpoint	@RestController	Path Operation Function
Dependency Injection	@Autowired	Depends()

Key Takeaway: If you understand layered architecture in Java, you already understand it in Python. The concepts are universal!



Step 1: The Data Contract (Schemas / DTOs)

A Pydantic Schema defines the "shape" of your API's JSON data. It's the direct equivalent of a Java DTO class used with `@RequestBody` and for responses.

Input Schema (AccountCreate)

For the request body

Output Schema (Account)

For the response body

```
# In schemas.py (like a dto package)
```

```
from pydantic import BaseModel
```

```
# Equivalent to an AccountCreatedDTO.java
```

```
class AccountCreate(BaseModel):  
    account_type: str  
    balance: float  
    customer_id: int
```

```
# Equivalent to an AccountResponseDTO.java
```

```
class Account(BaseModel):  
    id: int  
    account_number: str  
    account_type: str  
    balance: float  
    customer_id: int
```

```
class Config:
```

```
    # This tells Pydantic to be compatible with ORM models  
    # It's like using a mapping library (e.g., MapStruct)  
    orm_mode = True
```

Step 2: The Database Model (ORM Models / Entities)

An SQLAlchemy ORM Model represents a row in a database table. It's the direct equivalent of a JPA @Entity class.

```
# In models.py (like a domain or entity package)

from sqlalchemy import Column, Integer, String, Float, ForeignKey
from .database import Base

# Equivalent to a @Entity class
class BankAccount(Base):
    __tablename__ = "bank_accounts" # like @Table(name="...")

    id = Column(Integer, primary_key=True) # like @Id
    account_number = Column(String, unique=True, index=True, nullable=False)
    account_type = Column(String, nullable=False)
    balance = Column(Float, nullable=False)
    customer_id = Column(Integer, ForeignKey("customers.id")) # like @ManyToOne

# Dummy Customer model for the foreign key relationship
class Customer(Base):
    __tablename__ = "customers"
    id = Column(Integer, primary_key=True)
    name = Column(String)
```

Java Equivalent: This is exactly like defining a JPA Entity with @Entity, @Table, @Id, @Column, and @ManyToOne annotations.



Step 3: The "Brain" (Service Layer)

The Service Class contains the core business logic. It is decoupled from the web and database specifics. It's the direct equivalent of a Spring @Service class.

```
# In services.py (like a service package)
```

```
from sqlalchemy.orm import Session
from . import models, schemas
import random

class AccountService:
    # The DB session is injected, like injecting a Repository in Spring
    def __init__(self, db: Session):
        self.db = db

    def create_account(self, account_schema: schemas.AccountCreate) -> models.BankAccount:
        # 1. Business Logic: Generate a unique account number
        account_number = f"ACC{random.randint(1000000, 9999999)}"

        # 2. Map from Schema (DTO) to ORM Model (Entity)
        db_account = models.BankAccount(
            account_type=account_schema.account_type,
            balance=account_schema.balance,
            customer_id=account_schema.customer_id,
            account_number=account_number
        )

        # 3. Save to DB (this would be repository.save() in Spring)
        self.db.add(db_account)
        self.db.commit()
        self.db.refresh(db_account)
        return db_account
```

Java Equivalent: This is like a @Service class with @Autowired Repository dependency and business logic methods.



Step 4: Tying It All Together (Controller & DI)

The Path Operation Function is FastAPI's equivalent of a Spring @RestController method. Its job is to handle HTTP interaction and delegate to the service.

```
# In main.py (like a controller package)

from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session
from . import models, schemas, services
from .database import engine, get_db

# Create database tables
models.Base.metadata.create_all(bind=engine)

app = FastAPI()

@app.post("/accounts/", response_model=schemas.Account, status_code=201)
def create_new_account(
    account: schemas.AccountCreate,      # 1. Equivalent to @RequestBody
    db: Session = Depends(get_db)        # 2. DI, equivalent to @Autowired
):
    # 3. Basic validation (would be in service layer in larger apps)
    customer = db.query(models.Customer).filter(
        models.Customer.id == account.customer_id
    ).first()
    if not customer:
        raise HTTPException(
            status_code=404,
            detail=f"Customer with id {account.customer_id} not found."
        )

    # 4. Create service and delegate
    service = services.AccountService(db=db)
    return service.create_account(account_schema=account)
```

Java Equivalent: This is like a @RestController with @PostMapping, @RequestBody, and @Autowired dependencies.

Database Configuration & Dependency Injection

Setting up the database connection and DI mechanism (equivalent to Spring Boot's auto-configuration)

```
# In database.py

from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

# Use SQLite database for this example
SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

# This function is the dependency provider.
# FastAPI's Depends() will call this for each request.
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Java Equivalent: This is like Spring Boot's DataSource configuration with @Configuration and @Bean annotations for database connectivity.



Let's Get Building!

Your 30-Minute Challenge Starts Now!

1

Define your Pydantic Schemas

Create `schemas.py` with `AccountCreate` and `Account` models (equivalent to DTOs)



light_banking_api.zip

2

Create your SQLAlchemy ORM Model

Create `models.py` with `BankAccount` and `Customer` entities

3

Implement the AccountService

Create `services.py` with business logic (equivalent to `@Service`)

4

Create the FastAPI endpoint

Update `main.py` with the POST endpoint (equivalent to `@RestController`)

Pro Tip: Focus on the patterns, not the syntax. The architecture is the same as Java Spring Boot!



Conclusion & Key Takeaways



The Pattern is Universal

Controller → Service → Persistence is a robust, standard architecture in any modern backend framework. The concepts of DTOs/Schemas (for API data) and Entities/Models (for DB tables) are fundamental to clean, decoupled design.



Dependency Injection is Your Best Friend

DI (FastAPI's **Depends** or Spring's **@Autowired**) is the mechanism that enables this clean, layered architecture. It makes our code testable and maintainable.



You've Learned a New Stack, Not a New Paradigm

If you understand how to build a layered application in Java, you now understand how to do it in Python. **The principles are the same.**

What You Built Today

- Clean API endpoints with FastAPI
- Data validation with Pydantic
- Business logic in service layers
- Database operations with SQLAlchemy
- Dependency injection patterns

Skills Transferred from Java

- Layered architecture design
- Separation of concerns
- DTO/Entity patterns
- Dependency injection concepts
- RESTful API principles